

Feature: Gallery

From build-plan: feature 4

Branch: `feature/gallery`

Goal

`/gallery` and `/es/gallery` : every photo of her work, filterable by service, at a size where a cuticle line is actually visible. This is the page that sells the work, which is why it comes before the pages that describe it.

The page is built so that the honest small set she has today looks deliberate rather than unfinished, and so the same code carries thirty photos later without a rewrite.

Design reference

`prototypes/gallery.html` . Run it with `node prototypes/serve.mjs` ; it is not published as an artifact because every photograph in it is an Unsplash comp.

What to take from it: the masonry-by-CSS-columns grid at 2 / 3 / 4 columns, the sticky filter row of pill chips tinted by service accent, the count on the right, the hover scrim with a service tag, and the closing CTA band.

What **not** to take from it:

- Every photograph. All comps. See Notes.
- The `<h1>` "The work". It carries no Calgary, and the overview requires Calgary in the title, H1, copy and schema of every page. The `<Hot>` gradient-on-one-word pattern survives; the words change.
- The mockup-only warning note block.

In scope

- A `GalleryImage` record type and `data/gallery.ts` , following the keyed-against- the-dictionary pattern `data/faq.ts` already uses.
- Gallery copy in both dictionaries, including per-photo alt text.
- The `/gallery` route for both locales, with its own metadata, canonical and hreflang.
- ImageKit wiring: the URL endpoint env var and photos rendered through the `@imagekit/next` wrapper, plus the upload script that puts them in the library.
- The masonry grid, the service filter, and a live count.
- An honest empty state for when no records exist yet.

- `ImageGallery` JSON-LD built from the same records the grid renders.
- Lifting `Band` / `BandHead` / `Hot` and `BookingBand` out of `components/home/` so a second page can use them.

Out of scope

- **A lightbox or full-screen viewer.** The prototype has none; the grid is the size decision. If it turns out not to be, that is its own feature.
- **A home page gallery teaser.** Item 3's line, shipped without one because there were no photos. Adding it back is a decision of its own, once the photos land.
- **`aria-current` on the header nav.** Site-wide chrome, and nothing on this page breaks without it.
- **Per-service gallery URLs and a `?service=` query param.** See Data / contracts.
- **Sitemap, robots and `metadataBase`.** Item 9.
- **Uploading her photos to ImageKit.** Not code. See Open questions.
- **Service pages.** Item 5, even though the filter chips name the same three services.

Build loop

`blueprint/config.json` sets `workflow.stepReview: "feature"` and `workflow.checkpointCommits: "disabled"`. So: work the steps in order, no commits during implementation, and stop once at the end of the feature for review rather than after each step. `/complete` creates the single feature commit.

There is no declared Verify command in this project - `package.json` has no test runner and no `verify` script. The gate is `npm run build` (which runs TypeScript) and `npm run lint`, plus looking at the page. Baseline on `main` at spec time: build passes, two static routes, `/en` and `/es`.

Sequencing, as actually built. Steps 1 to 3 ran first, against no ImageKit account at all. Step 4 then split: 4a is the wiring, which needs only the endpoint, and 4b is her photographs, which nothing in the repo can supply. Steps 5 and 6 ran before 4b, out of numerical order and deliberately: every behaviour they own - reflow, filtering, counts, structured data - is independent of which photographs exist, and they were proved against temporary records that were reverted immediately. When the photos land, only `data/gallery.ts` and the `images` block in the two dictionaries change.

Build steps

- ☒ **1. Lift the page-section primitives out of `components/home/`.** Move `Band.tsx` (exporting `Band`, `BandHead`, `Hot`) and `BookingBand.tsx` to `components/site/`, and change `BookingBand` to take its copy as a prop instead of reading `t.home.booking` itself. Update the five importers. No behaviour change. *Why now:* the gallery needs all four, and items 5, 6 and 7 each need a closing CTA band too. Four copies by item 7 is the thing this prevents. **Done when:** `npm run build` and `npm run lint`

pass, no shared page-section primitive is left under `components/home/` (the home page importing its own sections from there is the only cross-directory import that should remain), and the home page renders identically to before at `/` and `/es`.

- ☒ **2. The record type, the empty data file, and the copy.** `data/gallery.ts` exporting a `GalleryImage` type and an empty `gallery` array, plus a `gallery` block in `dictionaries/en.ts` and its Spanish twin. Shapes in Data / contracts. **Done when:** `npm run build` passes, and deleting one key from the Spanish `gallery` block fails the build (that is the mechanism working, not a test to leave behind).
- ☒ **3. The route, the page head, the empty state and the CTA band.** `app/[locale]/gallery/page.tsx`. Eyebrow, H1, lede, the empty state, and the lifted `BookingBand` with gallery copy. No grid yet - with an empty records array the empty state is what renders, which is also the state the page will genuinely be in if it deploys before the photos land. **Done when:** the build route table lists `/en/gallery` and `/es/gallery` as prerendered, `/en/gallery` 301s to `/gallery`, `/gallery` and `/es/gallery` render the head, the empty state and the CTA band, and view-source shows this page's own `<link rel="canonical">` and three `hreflang` links pointing at `/gallery`, not at `/`.
- ☒ **4a. ImageKit wiring, the figure component and the upload script.** Ported from `face-and-body`, which is the same ImageKit account. `lib/imagekit.ts` holding `IMAGEKIT_FOLDER`, `MEDIA_VERSION` and a resolver that appends the folder when the endpoint lacks it; `components/gallery/GalleryFigure.tsx` using `Image` from `@imagekit/next`; `scripts/uploadImagesToImageKit.mjs`. **Done when:** `npm run build` and `npm run lint` pass, the resolver returns the same endpoint whether or not the configured value already carries the folder, a missing endpoint yields a labelled placeholder locally and throws only under `VERCEL=1`, and the upload script's `--dry-run` lists the exact remote paths it would write without sending anything.
- ☒ **4b. The real records.** Ten nail sets, cut from the 72 photographs she sent on 2026-09-21, uploaded to `facing-dani/nails` and recorded with alt text in both languages. Held back: two makeup shots and a portrait, because a model's face is recognisable in them and consent has not been asked - see Open questions. No lash photographs exist at all. **Done when:** every uploaded photo renders at `/gallery` and `/es/gallery` with its own alt text in that locale, the request goes to `ik.imagekit.io` under `/facing-dani/`, and nothing shifts as the images arrive.
- ☒ **5. The masonry grid and the service filter.** The grid at 2 / 3 / 4 columns, the chip row, the count. *Deviation, deliberate:* the mockup's chip row is `position: sticky; top: 58px`. Built non-sticky. The header's height is content-driven - no explicit height anywhere in `SiteHeader` - so there is no correct offset to stick to, and a guessed one puts photographs in the gap above the chips or hides the chips behind the header. Worth revisiting alongside giving the header a fixed height, and it matters more at thirty photos than at six. **Done when:** the grid reflows at the three breakpoints, each chip shows only that service and reads its accent when pressed, All restores every photo, the count matches what is on screen, a chip is reachable and operable by keyboard with a visible focus ring, a screen reader hears the new count after a chip is pressed, no chip renders for a service with no

photos, the chip row does not render at all when fewer than two services have photos, and with JavaScript disabled every photo still shows.

- ☒ **6. Structured data and the closing pass.** `imageGallerySchema` in `lib/schema.ts`, rendered through the existing `jsonLd` helper. Re-read the page copy against "nothing ships with placeholder text". **Done when:** the page emits one `ImageGallery` block listing every record with an absolute `ik.imagekit.io` `contentUrl`, it emits none at all when there are no records, and `npm run build` and `npm run lint` pass clean.

Files / areas

New:

- `app/[locale]/gallery/page.tsx` - server component, page metadata, JSON-LD
- `components/gallery/GalleryGrid.tsx` - "use client", the filter shell only
- `components/gallery/GalleryFigure.tsx` - server, one photo
- `data/gallery.ts`
- `lib/imagekit.ts` - ported from `face-and-body/lib/imagekitConfig.ts`
- `scripts/uploadImagesToImageKit.mjs` - ported from the sibling of the same name
- `.env.local` - untracked, and `NEXT_PUBLIC_IMAGEKIT_URL_ENDPOINT` also goes in the Vercel project. The overview records that there is deliberately no committed `.env.example`; do not add one.

Moved:

- `components/home/Band.tsx` -> `components/site/Band.tsx`
- `components/home/BookingBand.tsx` -> `components/site/BookingBand.tsx`

Changed:

- `components/home/Hero.tsx`, `ServiceCards.tsx`, `Story.tsx`, `Faq.tsx` - imports
- `app/[locale]/page.tsx` - imports, and pass `t.home.booking` into `BookingBand`
- `dictionaries/en.ts`, `dictionaries/es.ts` - the `gallery` block
- `lib/schema.ts` - `imageGallerySchema`
- `app/globals.css` - only if the filter's hide rule cannot be expressed as a Tailwind variant on the figure

`next.config.ts` is **not** changed. See Data / contracts.

Unchanged and worth knowing: `lib/locale.ts` already lists `/gallery` in `routes`, so `next.config.ts`'s rewrite and redirect for it exist. The header and footer already link to it. No routing change is needed beyond the page file.

Data / contracts

GalleryImage - shared fields in `data/gallery.ts`, keyed against the dictionary exactly as `data/faq.ts` keys the FAQ, so a record with no alt text is a compile error rather than an empty `alt`:

```
export type GalleryKey = keyof Dictionary["gallery"]["images"];

export type GalleryImage = {
  key: GalleryKey;
  /** Path within the ImageKit library, leading slash, no endpoint. */
  imagekitPath: string;
  serviceId: ServiceId;
  order: number;
  /** Intrinsic pixel dimensions of the source file. */
  width: number;
  height: number;
};
```

`width` and `height` are not in the overview's locked record, and they are required rather than optional: `@imagekit/next`'s `Image` wraps `next/image`, which needs intrinsic dimensions unless it is `fill`, and `fill` needs a fixed-aspect parent, which is the one thing a mixed-crop masonry grid cannot have.

Without them every photo pops in and shoves the column. They are shared, not locale-keyed, because a dimension does not translate.

Dictionary shape, added to both locales:

```
gallery.meta.title / .description    page metadata
gallery.eyebrow / .headingLead / .headingAccent / .lede
gallery.filters.label                aria-label for the chip group
gallery.filters.all                  the All chip
gallery.count.one                    "1 set"
gallery.count.other                  "{n} sets" - literal {n}, replaced at render
gallery.empty.heading / .body
gallery.cta.eyebrow / .headingLead / .headingAccent / .lede / .book / .instagram
gallery.images.<key>                 one alt string per photo
```

Chip labels for the three services reuse `t.services[id].name`. There is no second copy of "Nails".

`count` never renders zero: with no records the empty state replaces both the grid and the chip row, and a chip only exists for a service that has at least one photo.

Title. `gallery.meta.title` feeds the layout's `"%s - Dani Moreno"` template, so it must carry Calgary and the service terms itself and stay under roughly 60 characters including the suffix.

Metadata on the page, which must be set explicitly - the layout's `generateMetadata` points `canonical` and all three `hreflang` values at `/`, and a child's `alternates` replaces rather than merges with the

parent's, so a gallery page without this block declares the home page to be its canonical:

```
alternates: {
  canonical: localePath(locale, "/gallery"),
  languages: {
    en: localePath("en", "/gallery"),
    es: localePath("es", "/gallery"),
    "x-default": localePath(defaultLocale, "/gallery"),
  },
}
```

These stay relative. `metadataBase` is unset because the domain is not registered; that is item 9 and not this feature.

Env. Two variables, in the spelling the other client sites already use. Corrected from this spec's first draft after reading `primo-painters` and `face-and-body`, which share one ImageKit account with this project:

- `NEXT_PUBLIC_IMAGEKIT_URL_ENDPOINT` - the account endpoint with this project's folder on the end, `.../facing-dani`. Public by design: it is in every image URL.
- `IMAGE_KIT_PRIVATE_KEY` - server side only, read from `.env.local` at run time by the upload script and never shipped to the browser. The first draft said this key "must not be added". Wrong: the upload path is the script, and it is how both sibling projects put photos in the library. The spelling is `IMAGE_KIT_PRIVATE_KEY`, not the `IMAGEKIT_PRIVATE_KEY` the coding standards name; the convention already on disk wins.

The folder lives in code, not only in the endpoint. `IMAGEKIT_FOLDER = "facing-dani"` in `lib/imagekit.ts`, with a resolver that appends it when the configured endpoint does not already end with it, so either spelling works. `face-and-body/lib/imagekitConfig.ts` records why: production spent a day serving 404s because only one of two environments had the folder appended by hand.

A missing endpoint is not a local build failure. It renders a labelled placeholder box, and throws only under `process.env.VERCEL === "1"`. A local build without the endpoint is the account not existing yet; a deploy without it is a mistake.

`MEDIA_VERSION`, a date string on every ImageKit URL as a query parameter. ImageKit serves `max-age=31536000`, so a photo replaced under its existing filename keeps showing the old file to anyone who already loaded it, for a year. Bump it whenever a file is replaced in place. curl cannot diagnose this: it has no cache, so it reports the new image while every real browser shows the old one.

No `images.remotePatterns` entry, and no `images` config at all. Also corrected from the first draft. `@imagekit/next`'s `Image` supplies its own loader, so `next/image`'s remote-host allowlist never applies. `face-and-body` serves its entire catalogue this way with no `images` key in `next.config.ts`, and `primo-painters`' `remotePatterns` entry is for Vercel Blob, not ImageKit.

Image rendering. `Image` from `@imagekit/next`, and there is no choice about it. This spec's first draft said `@imagekit/next/server`, reasoning that a server-rendered image would keep the figures off the hydration path. That entry exports `getUploadAuthParams` and nothing else - the package ships a `dist/server/types/index.d.ts` that advertises an `Image`, but its exports map does not point at that file and the built module does not contain the component. `next build` catches it as `TS2305`. So each photo is a client boundary, exactly as in `face-and-body`; the figure, scrim and caption around it still render on the server, which is what the shell pattern is protecting.

`src` set to `imagekitPath`, `urlEndpoint` from `lib/imagekit.ts`, `width` and `height` from the record, `queryParameters={{ v: MEDIA_VERSION }}`, `className` giving `width: 100%; height: auto`. `sizes` must be set to match the column count or Next serves phone-sized images to a desktop and desktop-sized ones to a phone: `(min-width: 1000px) 25vw, (min-width: 620px) 33vw, 50vw`. The first four records get `priority`; the rest stay lazy.

Filter architecture. The page and every `<figure>` stay server components. The client shell owns exactly three things: which chip is pressed, a `data-filter` attribute on the grid container, and the count text. The figures are passed to it as `children` and visibility is a CSS attribute rule. This is the coding standards' "interactive shell in a client component, static content as `children`" rule, and the reason it applies here is concrete: the alternative hydrates every figure in order to toggle a class.

The shell also needs a per-service count map as a prop, since it cannot count children it does not read.

No filter state in the URL. No `?service=` and no `/gallery/nails`. Item 5 gives each service its own page targeting the same terms; a filtered gallery URL would compete with it, need its own canonical, and multiply this page by four in the sitemap at item 9. The filter is a view control, not a destination.

JSON-LD, built in `lib/schema.ts` from the same records and dictionary the grid renders, the way `faqPageSchema` already is, and serialised through the existing `jsonLd` helper:

```
{ "@context": "@type": "ImageGallery", name, description,  
  image: [{ "@type": "ImageObject", contentUrl, name: <alt> }] }
```

`contentUrl` is the endpoint joined to `imagekitPath`, absolute. Emitted only when there is at least one record.

Testing

No unit tests. This project has no test runner installed and `/tests` has not been run; adding one is not this feature's job. No browser test harness either, so nothing in this spec claims automated browser coverage.

The gate is `npm run build` (TypeScript runs inside it) and `npm run lint`, plus the manual checks named in each step's Done when. Two of those Done whens are deliberately destructive checks run once and reverted, not committed: deleting a Spanish key to prove the build fails, and unsetting the env var to prove the named error fires.

Check both locales every time. A gallery is the easiest page on this site to finish in English and forget in Spanish, because the photos look right either way.

Notes for the AI

No stock photography. Not one frame, not as a placeholder, not "until the real ones arrive." A gallery on a nail artist's site is a claim that this is her work. A client who books on a borrowed photo finds out in the chair. `links.md` records this as decided and `prototypes/home.html` said it before anyone asked. If the photo count is six, six ship - a small honest set reads as a new artist building up, which she is, and a big varied grid reads as a stock library, which people can tell.

This is also why the empty state is a real piece of work and not a stub. It is the most likely first state of this page.

No component contains the word Dani. The chips read from `services`, the copy from the dictionary, the business name from `siteConfig`. The template test applies to this page as much as any other.

Colour is navigation. Pink is nails, violet lashes, orange makeup, and the chip accent is keyed off `service.id` the way `ServiceCards` already does it, so the two cannot drift. Colour is never the only cue: every chip carries its name as text.

The accent fills fail contrast under 18px on light surfaces. Chip text on a filled chip uses `--on-hot`; the darker `--on-light` twins exist for the other case. Do not put 12px accent-fill text on a light background.

Hover does not exist on a phone, and the phone is the product. The prototype's `@media (hover: none)` rule that pins the scrim and tag on is not an afterthought; carry it over.

Animation is CSS. `motion` is not installed and is not being added.

Do not add a package. `@imagekit/next` is already a dependency and is the only thing this feature needs.

Open questions

1. ImageKit: resolved locally, still to do on Vercel. The account is the same one `primo-painters` and `face-and-body` already use. `.env.local` now holds `NEXT_PUBLIC_IMAGEKIT_URL_ENDPOINT` pointing at `.../facing-dani` and `IMAGE_KIT_PRIVATE_KEY`, both copied from the sibling. The `facing-dani` folder does not need creating by hand: the upload API creates any folder named in a request, which is how the sibling script has always worked.

Still outstanding: **the same two variables have to go into the Vercel project** for Production, Preview and Development. Until they do, a deploy renders placeholder boxes rather than photos, and the `VERCEL=1` guard throws.

2. Resolved: the photos were assessed on 2026-09-21. 72 images and 5 videos came down from Drive; 47 were iPhone HEIC. Ten nail sets shipped. The three things that came out of looking at all of them:

- **There are no lash photographs.** Not one, in 72. The site sells three services and the gallery can show two, and item 5's lashes page will have the same hole. This is a content ask for Dani, not a code problem.
- **Makeup is one look, two frames.** Both show the model's face clearly.
- **The shoot dates are July and August 2025,** so nothing here is recent work.

`blueprint/reference/photo-guide.html` covers most of what went wrong in the weaker shots - cluttered backgrounds, flat overhead light, framing too far back - but it has no rule about keeping other people out of the frame, which is the one failure that is a consent question rather than a composition one. Worth adding a fifth rule, noting the published artifact keeps its old content until republished.

4. Consent for the faces. Blocks the makeup chip. Three photographs are held out of `data/gallery.ts` pending a yes:

- two frames of a blue graphic-liner makeup look, model's face recognisable
- one portrait, almost certainly Dani herself, which belongs on item 6 anyway

Nothing with an identifiable face has been uploaded to ImageKit. The makeup pair is the only makeup in the set, so until they clear, `galleryServiceCounts()` returns one service and the filter row correctly does not render at all. Add them and the chip row appears with no code change.

3. What the tag on a photo says. Not blocking; spec'd as the service name. The mockup's tags are descriptive - "Marble tips", "Cat eye", "Volume set". This spec renders the service name instead, because the overview's locked `GalleryImage` has exactly one locale-keyed field and descriptive captions would need a second one plus two hand-written strings per photo, in a project where content is already the bottleneck. Adding `gallery.images.<key>.caption` later is additive and cheap. Say so now if the descriptive version is wanted from the start.

Status: verified against build output and the live CDN, not against a rendered page

`npm run lint` exit 0, `npm run build` exit 0. Route table gained `/en/gallery` and `/es/gallery`, both prerendered (SSG).

| Check | Result | |---|---| | Figures rendered | 10, all `data-service="nails"` | | Eager vs lazy | 4 eager, 6 `loading="lazy"` - matches `EAGER = 4` | | Chip row | absent, correctly: one service has photos, and the rule needs two | | Canonical | `/gallery` on EN, `/es/gallery` on ES - not the layout's `/` | | hreflang | 3 links per locale, all pointing at the gallery routes | | Title | carries Calgary and the service terms in both locales | | H1 | carries Calgary in both locales | | Alt text | per photo, per locale; Spanish strings render on `/es/gallery` | | JSON-LD | 10 `ImageObject` entries, absent entirely when records are empty | | Image URLs | `ik.imagekit.io/.../facing-dani/nails/<slug>.jpg?v=2026-09-21` | | CDN reachable | HTTP 200 on originals and on `tr=w-640` transforms | | Delivery resize | 293 KB master serves as 61 KB at 640px |

The missing-Spanish-key mechanism was re-proved. Deleting `gallery.filters.all` from `es.ts` failed the build:

```
dictionaries/es.ts(165,5): error TS2741: Property 'all' is missing in type  
'{ label: string; }' but required in type '{ label: string; all: string; }'
```

Restored.

The endpoint resolver was proved across five inputs: endpoint without the folder appends it; endpoint with the folder is unchanged; a trailing slash is normalized; unset yields a placeholder locally; unset under `VERCEL=1` throws `NEXT_PUBLIC_IMAGEKIT_URL_ENDPOINT is not set`.

Departures from the spec

1. `@imagekit/next/server` has no `Image`. The spec called for it, reasoning a server-rendered image would keep figures off the hydration path. That entry exports `getUploadAuthParams` and nothing else - the package ships a `dist/server/types/index.d.ts` advertising an `Image`, but its exports map does not point there and the built module does not contain it. `next build` caught it as `TS2305`. Each photo is now a client boundary, as in `face-and-body`; the figure, scrim and caption around it still render on the server. 2. **No `images.remotePatterns`, and no `images` config at all.** The spec required one for `ik.imagekit.io`. The wrapper supplies its own loader, so `next/image`'s remote allowlist never applies. `face-and-body` serves its whole catalogue with no `images` key; `primo-painters`' entry is for Vercel Blob. 3. **`IMAGEKIT_PRIVATE_KEY` was added**, which the spec's first draft forbade. The upload path is a script, and it is how both sibling projects put photos in the library. Note the spelling differs from the `IMAGEKIT_PRIVATE_KEY` in the coding standards; the convention already on disk won. 4. **The chip row is not sticky.** The mockup pins it at `top: 58px`. `SiteHeader` has no explicit height, so there is no correct offset, and a guessed one either gaps or hides the chips. Revisit alongside fixing the header height. 5. **"All" does not take the nails pink** the mockup gives it. One colour meaning two things breaks the locked rule that colour is navigation, so the All chip wears the brand gradient instead. 6. **Three sibling-project patterns were adopted** that the spec did not anticipate: `IMAGEKIT_FOLDER` in code with a tolerant resolver (a recorded day of production 404s on `face-and-body` came from the folder living only in an env var), `MEDIA_VERSION` as a cache-buster against ImageKit's one-year `max-age`, and a placeholder rather than a hard throw when the endpoint is missing locally.

Not verified

- **Nothing was seen rendered in a browser.** No dev server was started. The masonry reflow at 2/3/4 columns, the hover scrim, the focus ring, and the absence of layout shift are all argued from markup and CSS, not observed. Every image carries explicit `width / height`, so space is reserved by construction, but that is reasoning rather than evidence.

- **The filter has never been clicked.** Its behaviour was proved by building with five temporary records and reading the generated markup and CSS: all three `group-data-[filter=...]/grid:hidden` selectors exist in the production stylesheet, the count renders, the chip row disappears at one service. No human or browser has pressed a chip.
- `/en/gallery 301` to `/gallery` is argued from the `next.config.ts` rule and the identical rule already working for `/en`. Not observed.
- **Structured data was not run through Google's Rich Results test.** The claim in the spec was removed rather than asserted.
- **The Vercel environment still lacks both ImageKit variables.** A deploy today renders placeholder boxes and the `VERCEL=1` guard throws.

Implementation walkthrough

Lifting the page-section primitives

`Band`, `BandHead` and `Hot` were the page-section vocabulary for the whole site, but they lived in `components/home/` because the home page happened to be built first. The gallery needed all three, and items 5, 6 and 7 each need a closing CTA band too, so they moved to `components/site/` before anything else happened. `BookingBand` moved with them and stopped reading `t.home.booking` itself; it now takes a `BookingBandCopy` prop, which is what lets the gallery close on "Seen one you want?" while the home page still closes on "Let's do your set". Four importers were repointed and the import order fixed to the house grouping. No behaviour changed, which the prerendered home page confirmed.

The record, and why dimensions are required

`data/gallery.ts` follows `data/faq.ts` exactly: `GalleryKey` is derived from `Dictionary["gallery"]["images"]`, so a record whose alt text does not exist is a compile error rather than an empty `alt`. Adding a photo means adding its English alt first, which makes the build demand the Spanish one.

`width` and `height` are not in the overview's locked record and are not optional. `@imagekit/next`'s `Image` wraps `next/image`, which needs intrinsic dimensions unless it is `fill`, and `fill` needs a parent of fixed aspect - the one thing a mixed-crop masonry grid cannot have. Without them every photo pops in and shoves its column down the page. They are shared rather than locale-keyed because a dimension does not translate.

`galleryServiceCounts()` returns only services that actually have photos. That looked like defensive tidiness when it was written and turned out to be load-bearing: there are no lash photographs at all, and only one service has any, so the filter row correctly does not render.

ImageKit

`lib/imagekit.ts` is a port of `face-and-body/lib/imagekitConfig.ts`, same account, different folder. Two traps came across with it. `IMAGEKIT_FOLDER` lives in code and the resolver appends it when the configured endpoint lacks it, because on the sibling project production spent a day serving 404s when only one of two environments had the folder appended by hand - the resolver accepts both spellings so neither can be wrong. `MEDIA_VERSION` rides on every URL as a query parameter because ImageKit serves `max-age=31536000`, so a photo replaced under its existing filename keeps showing the old file for a year; curl cannot diagnose that, having no cache.

A missing endpoint is a labelled placeholder locally and a thrown error only under `VERCEL=1`. A local build without the variable is someone who has not pulled the env file; a deploy without it is a mistake.

`scripts/uploadImagesToImageKit.mjs` is the sibling's script with the folder changed and one addition: `--dry-run` no longer needs the private key, so the paths can be checked on a machine that has no credentials at all.

The grid, and where the interactivity stops

`GalleryGrid` is the only client component, and it owns three things: which chip is pressed, a `data-filter` attribute on the grid container, and the count text. The photographs arrive as `children`, already rendered on the server, and the hiding is pure CSS - each figure carries the two `group-data-[filter=...]` variants for the services that are *not* its own. Pressing a chip re-renders three buttons and a number, not the grid.

Those variant strings are written out as three literal entries in a `Record` rather than assembled, because Tailwind scans source text: a class name built at runtime is a class name it never generates. The production stylesheet was checked to confirm all three selectors exist.

Without JavaScript the chips render and do nothing and every photo stays visible, which is the right no-JS state for a gallery.

Structured data

`imageGallerySchema` reads the same records and dictionary the grid renders, so the markup and the schema cannot disagree - the same rule `faqPageSchema` already follows. It returns `null` in the two cases where there is nothing true to say: no photos, or no endpoint to address them by. `contentUrl` is the plain master URL with no `tr=` transformation and no `MEDIA_VERSION`, because both of those exist for the browser and neither belongs in an image's canonical address.

Choosing the photographs

72 stills came down from Drive, 47 of them iPhone HEIC. No ImageMagick, ffmpeg or Python on this machine, so decoding went through the Windows HEIF codec via WPF's `BitmapDecoder` in PowerShell, which also yielded each original's intrinsic size for the records. They were tiled into numbered contact sheets so the whole set could be judged at once, then the shortlist was re-rendered large from the originals to check sharpness before anything was chosen.

Ten nail sets were kept and ordered strongest first, so the four that load eagerly are the tight close-ups on dark backgrounds - which is also what the theme wants, since photo panels sit near [#1B1426](#). Masters were capped at 1500x2000 and re-encoded, taking 9.8 MB to 3.3 MB; the real saving is ImageKit's delivery transform, which serves a 293 KB master as 61 KB at 640px.

Nothing with an identifiable face was uploaded. The only makeup in the set is one look in two frames, both showing the model clearly, so it is held back along with a portrait of Dani that belongs on item 6 regardless.